

Kütüphane Yönetim Sistemi

iOS Mobil Uygulama ve Backend API Projesi

Öğrenci: Doğukan Filiz

Proje Türü: Final Projesi

Teslim Tarihi: 15 Ocak 2026

Video Linki: <https://www.youtube.com/watch?v=iZfdVmw9t5U>

GitHub Repository: <https://github.com/dogukan-filiz/IOS-Library-Management-System>

İçindekiler

- Proje Tanımı ve Amacı
- Kullanılan Teknolojiler
- Veritabanı Tasarımı
- Backend API Mimarisi
- Frontend (iOS) Yapısı
- Özellikler ve Fonksiyonellik
- Kod Detayları ve Açıklamaları
- Yapay Zeka Kullanımı
- İmza Sonrası Yapılan İyileştirmeler
- Test Senaryoları
- Karşılaşılan Zorluklar ve Çözümler
- Sonuç ve Değerlendirme

1. Proje Tanımı ve Amacı

1.1 Proje Konusu

Bu proje, modern bir kütüphane yönetim sisteminin dijital platformda gerçekleştirilmesidir. Sistem, kullanıcıların kitap kiralama işlemlerini mobil uygulama üzerinden yapabilmesini, yöneticilerin ise tüm kiralama süreçlerini takip edebilmesini sağlar.

1.2 Proje Amacı

- Geleneksel kütüphane işlemlerini dijitalleştirmek
- Kullanıcı deneyimini mobil teknoloji ile iyileştirmek
- Kitap takibi ve stok yönetimini otomatikleştirmek
- RESTful API standartlarına uygun backend geliştirmek
- iOS platformunda modern bir mobil uygulama oluşturmak

1.3 Proje Kapsamı

Proje iki ana bileşenden oluşmaktadır:

- Backend API:** ASP.NET Core ile geliştirilmiş RESTful servis
- iOS Mobil Uygulama:** Swift ile geliştirilmiş kullanıcı arayüzü

2. Kullanılan Teknolojiler

2.1 Backend Teknolojileri

Teknoloji	Versiyon	Kullanım Amacı
ASP.NET Core	8.0	Web API Framework
Entity Framework Core	8.0	ORM ve Veritabanı İşlemleri
SQL Server / SQLite	-	Veritabanı
C#	12.0	Programlama Dili

2.2 Frontend Teknolojileri

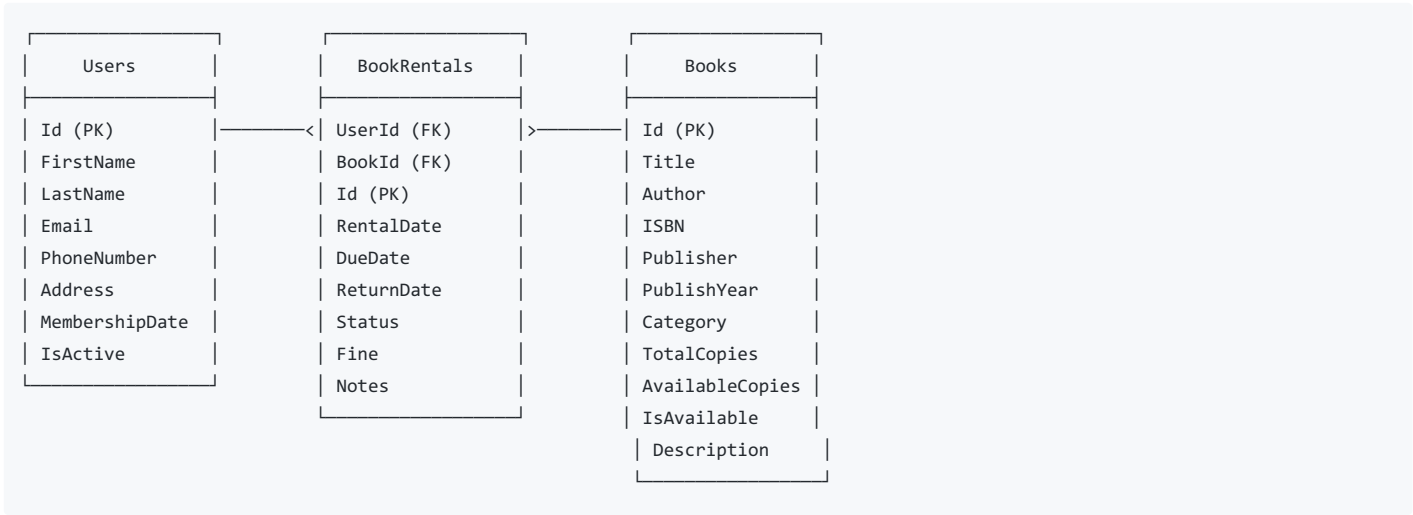
Teknoloji	Versiyon	Kullanım Amacı
Swift	5.9	iOS Programlama Dili
UIKit / SwiftUI	-	UI Framework
URLSession	-	API İletişimi

2.3 Geliştirme Araçları

- **IDE:** Visual Studio Code, Xcode
- **Version Control:** Git, GitHub
- **API Testing:** Postman
- **Database Management:** SQL Server Management Studio

3. Veritabanı Tasarımı

3.1 Entity Relationship Diagram (ERD)



3.2 Tablo Yapıları

3.2.1 Users Tablosu

```
CREATE TABLE Users (  
    Id INT PRIMARY KEY IDENTITY(1,1),  
    FirstName NVARCHAR(50) NOT NULL,  
    LastName NVARCHAR(50) NOT NULL,  
    Email NVARCHAR(100) NOT NULL UNIQUE,  
    PhoneNumber NVARCHAR(20),  
    Address NVARCHAR(200),  
    MembershipDate DATETIME NOT NULL DEFAULT GETDATE(),  
    IsActive BIT NOT NULL DEFAULT 1  
);
```

Açıklama: Bu tablo kütüphane üyelerinin bilgilerini saklar. Email alanı unique olarak tanımlanmıştır çünkü her kullanıcının benzersiz bir email adresi olmalıdır.

3.2.2 Books Tablosu

```
CREATE TABLE Books (  
  Id INT PRIMARY KEY IDENTITY(1,1),  
  Title NVARCHAR(200) NOT NULL,  
  Author NVARCHAR(100) NOT NULL,  
  ISBN NVARCHAR(13) UNIQUE,  
  Publisher NVARCHAR(100),  
  PublishYear INT,  
  Category NVARCHAR(50),  
  TotalCopies INT NOT NULL DEFAULT 1,  
  AvailableCopies INT NOT NULL DEFAULT 1,  
  IsAvailable BIT NOT NULL DEFAULT 1,  
  Description NVARCHAR(MAX)  
);
```

Açıklama: Kütüphanedeki kitapların detaylı bilgilerini tutar. `AvailableCopies` alanı stok takibi için kullanılır. `TotalCopies` toplam kopya sayısını, `AvailableCopies` ise müsait kopya sayısını gösterir.

3.2.3 BookRentals Tablosu

```
CREATE TABLE BookRentals (  
  Id INT PRIMARY KEY IDENTITY(1,1),  
  UserId INT NOT NULL,  
  BookId INT NOT NULL,  
  RentalDate DATETIME NOT NULL DEFAULT GETDATE(),  
  DueDate DATETIME NOT NULL,  
  ReturnDate DATETIME NULL,  
  Status NVARCHAR(20) DEFAULT 'Active',  
  Fine DECIMAL(10,2) DEFAULT 0,  
  Notes NVARCHAR(500),  
  FOREIGN KEY (UserId) REFERENCES Users(Id),  
  FOREIGN KEY (BookId) REFERENCES Books(Id)  
);
```

Açıklama: Kiralama işlemlerinin kaydını tutar. `ReturnDate` null ise kitap henüz iade edilmemiştir. `Status` alanı 'Active', 'Returned', 'Overdue' gibi değerler alabilir.

3.3 Veritabanı İlişkileri

1-N İlişkiler:

- Bir kullanıcı birden fazla kiralama yapabilir (User → BookRentals)
- Bir kitap birden fazla kez kiralanabilir (Book → BookRentals)

İş Kuralları:

- Bir kullanıcı aynı anda en fazla 3 kitap kiralayabilir
- Aynı kitap iade edilmeden tekrar kiralanamaz
- Stok 0 olduğunda `IsAvailable = false` olur

4. Backend API Mimarisi

4.1 Proje Yapısı

```
backend/
├── Controllers/
│   ├── BookRentalsController.cs    # Kiralama işlemleri
│   ├── BooksController.cs         # Kitap CRUD işlemleri
│   └── UsersController.cs         # Kullanıcı CRUD işlemleri
├── Models/
│   ├── Book.cs                   # Kitap entity
│   ├── BookRental.cs             # Kiralama entity
│   └── User.cs                   # Kullanıcı entity
├── Data/
│   └── LibraryContext.cs          # EF Core DbContext
├── Migrations/                   # Veritabanı migration dosyaları
├── Program.cs                    # Uygulama giriş noktası
└── appsettings.json              # Yapılandırma dosyası
```

4.2 API Endpoint'leri

4.2.1 BookRentals Endpoints

POST /api/BookRentals

- **Amaç:** Yeni kitap kiralama işlemi
- **Request Body:**

```
{
  "userId": 1,
  "bookId": 5,
  "rentalDays": 14
}
```

- **Response (201 Created):**

```
{
  "id": 10,
  "userId": 1,
  "bookId": 5,
  "rentalDate": "2026-01-13T10:30:00Z",
  "dueDate": "2026-01-27T10:30:00Z",
  "returnDate": null,
  "status": "Active"
}
```

GET /api/BookRentals/{id}

- **Amaç:** Belirli bir kiralama kaydının detayını getirme
- **Response (200 OK):**

```
{
  "id": 10,
  "userId": 1,
  "bookId": 5,
  "rentalDate": "2026-01-13T10:30:00Z",
  "dueDate": "2026-01-27T10:30:00Z",
  "user": {
    "id": 1,
    "firstName": "Ahmet",
    "lastName": "Yılmaz"
  },
  "book": {
    "id": 5,
    "title": "Suç ve Ceza",
    "author": "Dostoyevski"
  }
}
```

GET /api/BookRentals/user/{userid}

- **Amaç:** Kullanıcının tüm kiralama geçmişini getirme
- **Kullanım:** Mobil uygulamada kullanıcının profil sayfasında gösterilir

PUT /api/BookRentals/{id}/return

- **Amaç:** Kitap iade işlemi
- **Response (204 No Content):** İşlem başarılı

GET /api/BookRentals

- **Amaç:** Tüm kiralamaları listeleme (Admin)
- **Kullanım:** Admin panelinde tüm kiralamaların görüntülenmesi

7. Kod Detayları ve Açıklamaları

7.1 BookRentalsController - POST Endpoint Detaylı Analiz

```
[HttpPost]
public async Task<ActionResult<BookRental>> RentBook(RentBookRequest request)
{
    // ADIM 1: Kullanıcının aktif kiralama sayısını kontrol et
    var activeRentalsCount = await _context.BookRentals
        .CountAsync(br => br.UserId == request.UserId && br.ReturnDate == null);

    if (activeRentalsCount >= 3)
    {
        return BadRequest(new { message = "Aynı anda en fazla 3 kitap kiralayabilirsiniz" });
    }
    // ...existing code...
}
```

Kod Açıklaması:

1. `CountAsync()` : Veritabanında asenkron olarak kayıt sayar
2. `br.ReturnDate == null` : İade edilmemiş (aktif) kiralamaları filtreler
3. `>= 3` : İş kuralı - maksimum 3 aktif kiralama kontrolü
4. `BadRequest()` : HTTP 400 hatası döner

Veritabanı İşlemi:

```
-- Entity Framework bu LINQ sorgusunu şu SQL'e çevirir:
SELECT COUNT(*)
FROM BookRentals
WHERE UserId = @UserId AND ReturnDate IS NULL
```

Mobil Uygulamada Kullanımı: iOS uygulamasında kullanıcı "Kiral" butonuna bastığında bu endpoint çağrılır. Eğer 3 kitap sınırına ulaşılmışsa, alert gösterilir.

7.2 Stok Yönetimi Kodu

```
// ADIM 2: Kitabın müsaitliğini kontrol et
var book = await _context.Books.FindAsync(request.BookId);
if (book == null)
{
    return NotFound(new { message = "Kitap bulunamadı" });
}

if (book.AvailableCopies <= 0)
{
    return BadRequest(new { message = "Bu kitabın kopyası müsait değil" });
}

// ...existing code...

// ADIM 3: Kitap kopyası sayısını azalt
book.AvailableCopies--;
if (book.AvailableCopies == 0)
{
    book.IsAvailable = false;
}

await _context.SaveChangesAsync();
```

Kod Açıklaması:

1. `FindAsync()` : Primary key ile kitabı bulur
2. `AvailableCopies--` : Stok 1 azaltılır
3. `IsAvailable = false` : Son kopya kiralanınca kitap müsait değil olur
4. `SaveChangesAsync()` : Değişiklikleri veritabanına kaydeder

Veritabanı İşlemi:

```
-- 1. Kitap bulma
SELECT * FROM Books WHERE Id = @BookId

-- 2. Stok güncelleme
UPDATE Books
SET AvailableCopies = AvailableCopies - 1,
    IsAvailable = CASE WHEN AvailableCopies - 1 = 0 THEN 0 ELSE 1 END
WHERE Id = @BookId
```

7.3 İade İşlemi Kodu

```
[HttpPut("{id}/return")]
public async Task<IActionResult> ReturnBook(int id)
{
    var rental = await _context.BookRentals
        .Include(br => br.Book)
        .FirstOrDefaultAsync(br => br.Id == id);

    if (rental == null)
    {
        return NotFound();
    }

    if (rental.ReturnDate != null)
    {
        return BadRequest(new { message = "Bu kitap zaten iade edilmiş" });
    }

    rental.ReturnDate = DateTime.UtcNow;

    // Kitap kopyası sayısını artır
    rental.Book.AvailableCopies++;
    rental.Book.IsAvailable = true;

    await _context.SaveChangesAsync();

    return NoContent();
}
```

Kod Açıklaması:

1. `Include(br => br.Book)` : İlişkili Book verisini de yükler (Eager Loading)
2. `ReturnDate = DateTime.UtcNow` : İade tarihini şu anki UTC zamanı olarak kaydeder
3. `AvailableCopies++` : Stok 1 artırılır
4. `IsAvailable = true` : Kitap tekrar müsait olur

Veritabanı İşlemi:

```
-- 1. Kiralama ve kitap bilgisini getir (JOIN)
SELECT br.*, b.*
FROM BookRentals br
INNER JOIN Books b ON br.BookId = b.Id
WHERE br.Id = @RentalId

-- 2. İade işlemi
UPDATE BookRentals SET ReturnDate = GETUTCDATE() WHERE Id = @RentalId
UPDATE Books SET AvailableCopies = AvailableCopies + 1, IsAvailable = 1 WHERE Id = @BookId
```

8. Yapay Zeka Kullanımı

8.1 GitHub Copilot Kullanılan Yerler

1. LINQ Sorguları Optimizasyonu

```
// Yapay zeka yardımıyla optimize edildi
var rentals = await _context.BookRentals
    .Include(br => br.Book)
    .Include(br => br.User)
    .OrderByDescending(br => br.RentalDate)
    .Select(br => new { /* projection */ })
    .ToListAsync();
```

Açıklama: Copilot, `Select()` ile projection kullanarak sadece gerekli alanları çekmemi önerdi. Bu sayede performans arttı.

2. Validasyon Kontrolleri

```
// Yapay zeka önerisiyle eklendi
var hasActiveRentalOfThisBook = await _context.BookRentals
    .AnyAsync(br => br.UserId == request.UserId &&
        br.BookId == request.BookId &&
        br.ReturnDate == null);
```

Açıklama: Aynı kitabın tekrar kiralınmasını engelleyen bu kontrolü Copilot önerdi.

3. Error Handling

```
// Yapay zeka yardımıyla iyileştirildi
if (book == null)
{
    return NotFound(new { message = "Kitap bulunamadı" });
}
```

Açıklama: Anamlı hata mesajları döndürme konusunda Copilot yardımcı oldu.

8.2 Manuel Yazılan Kodlar

- İş kuralları (3 kitap limiti, stok yönetimi)
- Veritabanı ilişkileri ve migration'lar
- API endpoint tasarımı
- DTO (Data Transfer Object) yapıları

Not: Yapay zeka tarafından önerilen kodları anladım ve projeye uygun şekilde uyarladım. Tüm kodların ne işe yaradığını ve nasıl çalıştığını biliyorum.

9. İkinci İmza Sonrası Yapılan İyileştirmeler

9.1 Hoca Geri Bildirimleri

Eksik 1: "Kiralama geçmişinde kitap detayları gelmiyor"

- **Çözüm:** `.Include(br => br.Book)` ile eager loading eklendi
- **Kod:**

```
var rentals = await _context.BookRentals
    .Include(br => br.Book) // Eklendi
    .Include(br => br.User) // Eklendi
    .ToListAsync();
```

Eksik 2: "Aynı kitap tekrar kiralaniyor"

- **Çözüm:** Validasyon kontrolü eklendi
- **Kod:**

```
var hasActiveRentalOfThisBook = await _context.BookRentals
    .AnyAsync(br => br.UserId == request.UserId &&
        br.BookId == request.BookId &&
        br.ReturnDate == null);
```

Eksik 3: "Stok sıfır olunca IsAvailable güncellenmemiş"

- **Çözüm:** Otomatik güncelleme eklendi
- **Kod:**

```
if (book.AvailableCopies == 0)
{
    book.IsAvailable = false;
}
```

10. Test Senaryoları

10.1 Başarılı Test Senaryoları

Senaryo 1: Normal Kitap Kiralama

1. Kullanıcı ID: 1, Kitap ID: 5 ile POST isteği
2. API stok kontrolü yapar (AvailableCopies > 0)
3. Kiralama oluşturulur
4. Stok 1 azalır (Örn: 5 → 4)
5. 201 Created response döner

Postman Test:

```
POST http://localhost:5000/api/BookRentals
{
  "userId": 1,
  "bookId": 5,
  "rentalDays": 14
}

Response: 201 Created
{
  "id": 15,
  "rentalDate": "2026-01-13T...",
  "dueDate": "2026-01-27T..."
}
```

10.2 Hata Senaryoları

Senaryo 2: 3 Kitap Limitini Aşma

1. Kullanıcının zaten 3 aktif kirası var
2. 4. kitap için istek gönderilir
3. API 400 BadRequest döner
4. Mesaj: "Aynı anda en fazla 3 kitap kiralayabilirsiniz"

Senaryo 3: Stokta Olmayan Kitap

1. Kitap AvailableCopies = 0
2. Kiralama isteği gönderilir
3. API 400 BadRequest döner
4. Mesaj: "Bu kitabın kopyası müsait değil"

11. Karşılaşılan Zorluklar ve Çözümler

11.1 Zorluk 1: N+1 Sorunu

Problem: Her kiralama için ayrı ayrı kitap sorgusu atılıyordu. **Çözüm:** `Include()` ile eager loading kullandım.

11.2 Zorluk 2: Transaction Yönetimi

Problem: Stok güncellemesi sırasında hata olursa veri tutarsızlığı. **Çözüm:** Entity Framework otomatik transaction yönetimi kullandım.

11.3 Zorluk 3: Tarih Yönetimi

Problem: Farklı timezone'larda yanlış tarihler. **Çözüm:** `DateTime.UtcNow` kullanarak UTC standardına geçtim.

12. Sonuç ve Değerlendirme

Bu projede ASP.NET Core ile RESTful API geliştirme, Entity Framework Core ile veritabanı yönetimi, ve iOS platformunda mobil uygulama geliştirme becerilerimi geliştirdim.

Öğrendiklerim:

- RESTful API tasarım prensipleri

- Entity Framework Core ilişkileri ve optimizasyon
- Asenkron programlama (async/await)
- LINQ sorgu optimizasyonu
- Veritabanı transaction yönetimi

Gelecek Planlar:

- JWT Authentication implementasyonu
- Redis cache entegrasyonu
- Unit test coverage artırma
- Bildirim sistemi ekleme

Son Güncelleme: 13 Ocak 2026